A Smart Attendance System

Major Project Submitted
In
Partial fulfillment of the requirement of the degree

Of

Bachelor of Technology
in
Computer Science and Engineering

By

Soumen Rakshit –-27600122039
Tanmoy khanra – 27600122129
Divyanshu Singh – 27600122098
Vipin Kumar – 27600122154
Vikash Kumar - 27600122230

Under the guidance of
Prof. Nivedita Das
(Department of Computer Science and Engineering)

BUDGE BUDGE INSTITUTE OF TECHNOLOGY

Affiliated to MAKAUT (formerly WBUT) and Approved by AICTE, Accredited by- NAAC, NBA(National Board of Accreditation-CSE,EE,ECE,ME) valid from Academic Year 2022-23to2025-2026 i.e.upto 30-07-2026 http://www.bbit.edu.in

July2026

<u>DECLARATION</u>

We certify that:

a. The project report entitled **"A Smart Attendance System"** is our original work carried out under the guidance of **Prof. Nivedita Das**. This work has not been submitted to any other institution or university for the award of any degree .

b. We have followed the guidelines provided by the Institute in preparing this project report and have conformed to the prescribed ethical norms and academic standards while carrying out the work and writing the report.

c. Whenever we have used materials from other sources, we have given due credit by citing them appropriately in the report, providing complete references, and obtaining permission from the respective copyright holders whenever necessary.

Name and Signature of
Soumen Rakshit
UniversityRollNo**:27600122039**

Name and Signature of
Tanmoy Khanra
UniversityRollNo**:27600122129**

Name and Signature of
Divyanshu Singh
UniversityRollNo**:27600122098**

Name and Signature of
Vipin Kumar
UniversityRollNo**:27600122154**

Name and Signature of
Vikash Kumar
UniversityRollNo**:27600122230**

ACKNOWLEDGEMENT

We would like to express our sincere gratitude to our project guide **Prof. Nivedita Das** for his valuable guidance, continuous support, and encouragement throughout the development of this project. His insights, suggestions, and technical expertise played a crucial role in the successful completion of our work.

We are also thankful to the **Department of Computer Science and Engineering**, **Budge Budge Institute of Technology,** for providing the necessary resources and a supportive academic environment. We would like to extend our gratitude to all faculty members, friends, and our parents for their constant support and motivation during the course of this project.

PROJECT STUDENTS
SOUMEN RAKSHIT (27600122039)
TANMOY KHANRA (27600122129)
DIVYANSHU SINGH (27600122098)
VIPIN KUMAR (27600122154)
VIKASH KUMAR (27600122230)

CERTIFICATE

This is to certify that the project entitled **"A Smart Attendance System"** has been carried out by the students of the Department of Computer Science & Engineering, **Budge Budge Institute of Technology (BBIT)**, affiliated to **Maulana Abul Kalam Azad University of Technology (MAKAUT)**, West Bengal, in partial fulfillment of the requirements for the award of the degree of **Bachelor of Technology (B.Tech)** in Computer Science & Engineering during the academic year **2025–2026**.

The project work has been completed under my supervision and guidance, and to the best of my knowledge, the work presented in this project is original and has not been submitted elsewhere for the award of any degree or diploma.

Submitted by:

1. **Soumen Rakshit** (URN: 27600122039)
2. **Tanmoy Khanra** (URN: 27600122129)
3. **Divyanshu Singh** (URN: 27600122098)
4. **Vipin Kumar** (URN: 27600122154)
5. **Vikash Kumar** (URN: 27600122230)

Guided by:

Prof. Nivedita Das

(Department of Computer Science & Engineering)

Budge Budge Institute of Technology

______________________________ ______________________________

Signature of Guide **Signature of H.O.D.**

Date: ________________

Place: Budge Budge Institute of Technology, Kolkata

NO-PLAGIARISM DECLARATION

Title of the Project: A Smart Attendance System

Name(s) of the learner: Soumen Rakshit,Tanmoy Khanra, Divyanshu Singh, Vipin Kumar, Vikash Kumar

PRN:

Name of Project Supervisor: Prof. Nivedita Das

I declare that this Project Report is the result of my independent work. All sources of information have been duly acknowledged. No portion of this work has been copied or reproduced from any other project or publication. I understand that plagiarism is a serious offence and may result in rejection of my work and appropriate action against me.

Signature of Student

Date:

Place: Budge Budge

ABSTRACT

Traditional attendance marking methods, such as roll calls or paper-based signing sheets, are time-consuming, prone to human error, and susceptible to "proxy" attendance. In an era of digitalization, these methods create inefficiencies in classroom management and academic monitoring.

This project, **"Smart Attendance System,"** proposes an automated solution leveraging Biometric Facial Recognition technology. The system captures live video feed from a classroom camera, detects faces, and matches them against a pre-enrolled database using deep learning algorithms (**VGG-Face** model). Upon successful identification, the student's attendance is automatically logged for the specific class period based on a real-time scheduling engine.

The system is built using a **Microservices Architecture**. The frontend is developed with **React.js** for a responsive user interface, while the backend is split into two distinct services: a **Node.js/Express** server handling business logic, scheduling, and database operations, and a **Python/Flask** server handling heavy AI/Computer Vision tasks using the **DeepFace** library. Data is persisted securely in a **MongoDB** database.

Key features include real-time burst-mode scanning, period-wise tracking, automated schedule detection, and comprehensive reporting with Excel export capabilities. This solution eliminates manual effort, ensures data accuracy, and provides actionable insights into student attendance patterns.

TABLE OF CONTENT

LIST OF FIGURES

LIST OF TABLES

CHAPTER 1

INTRODUCTION

Attendance management is an important academic activity used to monitor the regularity and participation of students in educational institutions. Traditional attendance systems, such as manual registers and spreadsheets, are time-consuming and prone to errors. These methods also allow issues like proxy attendance and difficulty in maintaining large records.

The Smart Attendance System is a modern, technology-based solution designed to automate and simplify the attendance process. It helps teachers and administrators record, manage, and analyze attendance efficiently, while students can easily view their attendance status.The system reduces manual effort and improves accuracy and transparency.

The system is developed using modern web technologies and follows a client–server architecture. The frontend provides a user-friendly interface, while the backend securely processes and stores attendance data in a centralized database. This ensures data reliability, fast access, and easy maintenance.

The main objective of the Smart Attendance System is to eliminate the drawbacks of traditional attendance methods and provide a secure, efficient, and reliable attendance management solution. It is suitable for schools and colleges and helps improve overall administrative efficiency.

1.1 Overview of Smart Attendance System:

The Smart Attendance System is an automated software solution designed to manage and record student attendance digitally. It replaces traditional manual attendance methods with a more efficient, accurate, and secure system. The system allows teachers to mark attendance electronically, administrators to manage student and subject records, and students to view their attendance details.

The system is developed using modern web technologies and operates on a client–server architecture. Users interact with the system through a web-based interface, while all attendance data is processed and stored securely in a centralized database. This ensures fast data access, data consistency, and easy maintenance.

The Smart Attendance System helps reduce human errors, prevents proxy attendance, and saves time during class sessions. It also supports the generation of attendance reports, such as daily, monthly, and subject-wise reports, which are useful for academic monitoring and decision- making. 1

Overall, the Smart Attendance System provides a reliable and user-friendly solution for educational institutions to efficiently manage attendance and improve administrative processes.

1.2 Problem Statement of Traditional Attendance System:

Traditional attendance systems used in schools and colleges are mostly manual and paper-based. These methods require a significant amount of time to record attendance and are prone to human errors. Maintaining large attendance registers becomes difficult, and searching or updating records is inefficient.

Another major problem with traditional systems is proxy attendance, where students mark attendance on behalf of others. This reduces the reliability and accuracy of attendance records. Additionally, manual systems do not provide real-time access to attendance data, making report generation and analysis a slow process.

As the number of students increases, managing attendance manually becomes more complex and unorganized. There is also a risk of data loss, damage to records, and lack of proper security.
These challenges highlight the need for an automated, secure, and efficient attendance management system.

The Smart Attendance System is proposed to address these issues by providing a digital solution that ensures accuracy, transparency, security, and efficient attendance management.

1.3 Objectives of the Project:

The main objective of the Smart Attendance System is to automate and simplify the attendance management process in educational institutions. The specific objectives of the project are as follows:

- To eliminate manual attendance marking and reduce human errors
- To prevent proxy attendance and improve accuracy
- To provide a secure and reliable system for attendance management
- To enable easy storage and retrieval of attendance data
- To allow teachers to mark and manage attendance efficiently
- To provide students with access to their attendance records
- To generate attendance reports such as daily, monthly, and subject-wise reports
- To reduce administrative workload and save time

1.4 Flow Chart:

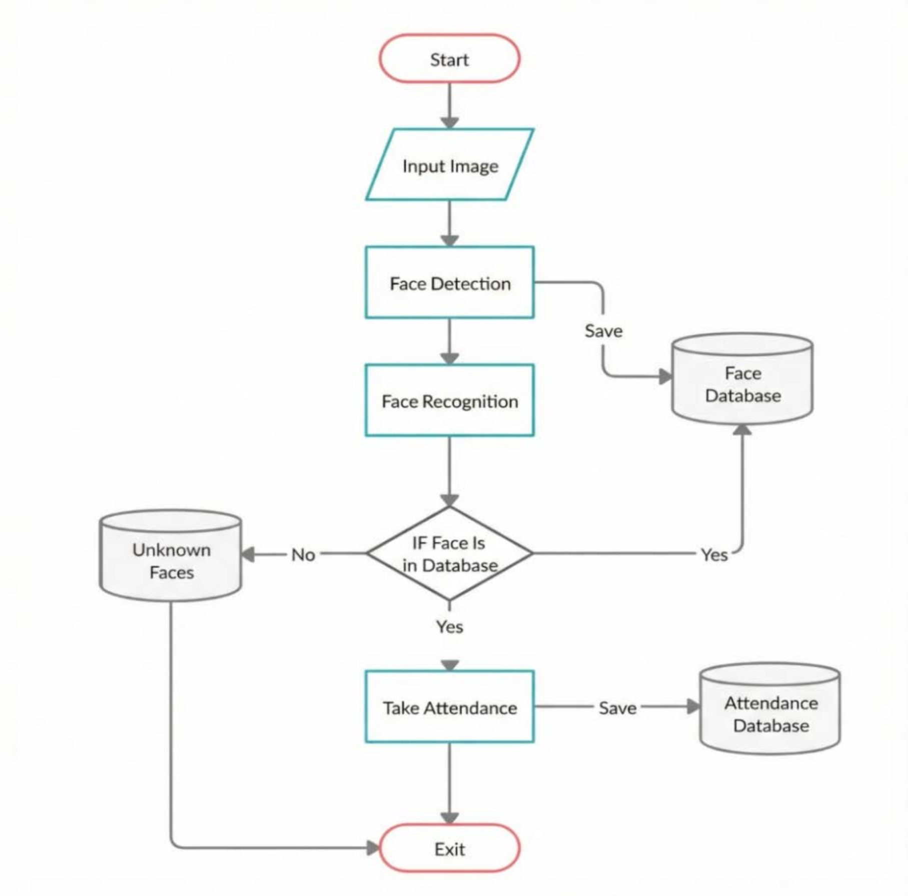


Figure 1 Flow Chart

1.5 Scope of the Project:

The scope of the Smart Attendance System includes the development of a digital platform to manage and record student attendance efficiently. The system is designed to be used in educational institutions such as schools and colleges for daily attendance management.

The project covers features such as student registration, subject management, digital attendance marking, attendance tracking, and report generation. It allows teachers and administrators to manage attendance data in a secure and organized manner, while students can view their attendance details.

The system supports multiple users with role-based access, including Admin, Teacher, and Student. It is designed to handle a large number of records and users, making it suitable for institutions of different sizes.

The Smart Attendance System focuses on improving accuracy, reducing manual effort, and providing reliable attendance records. Future enhancements such as biometric integration, QR code attendance, and mobile application support can be added to expand the system's functionality.

CHAPTER 2

LITERATURE REVIEW

2.1 Review of Existing Techniques

Rathor et al. (2012) discussed the traditional method of manual attendance, noting that it is highly time-consuming and prone to human error. They emphasized that calling out names or passing signature sheets often takes 5–10 minutes per lecture, distracting from teaching time. Furthermore, this method is fundamentally flawed as it easily allows for "proxy attendance," where students sign on behalf of their absent peers.

To address these issues, hardware-based solutions were introduced. Lim et al. (2015) proposed an RFID (Radio Frequency Identification) based system due to its simplicity and speed. However, they acknowledged a critical disadvantage: the system cannot verify the identity of the cardholder. A student can easily carry a friend's ID card to mark them present, rendering the system ineffective against unauthorized attendance. Similarly, Saraswat et al. (2016) implemented a fingerprint recognition system. While this method effectively eliminates the issue of proxy attendance, they noted that it is not efficient for large classes. The verification process requires physical contact with the sensor, forcing students to wait in long queues, which causes congestion at the classroom entrance and potential hygiene concerns.

2.2 Evolution of Face Recognition:

In the domain of computer vision, Viola and Jones (2001) revolutionized object detection with their Haar Cascade algorithm. Early face recognition systems relied on such handcrafted features and techniques like Local Binary Patterns Histograms (LBPH). However, Arun Katara et al. (2017) pointed out that while face recognition is less intrusive than iris scanning and more accurate than voice recognition, early algorithms struggled significantly with real-world conditions. These systems frequently failed when subjects were not looking directly at the camera or when lighting conditions varied, making them unreliable for institutional use.

2.3 The Deep Learning Approach:

Recent advancements have shifted towards Deep Learning to overcome these accuracy limitations. Parkhi et al. (2015) introduced the VGG-Face model, demonstrating that Convolutional Neural Networks (CNNs) trained on massive datasets could achieve near-human accuracy even with pose and lighting variations. Building on this, Serengil and Ozpinar (2020) presented the DeepFace framework, which hybridizes state-of-the-art models like VGG-Face and Google FaceNet. They concluded that this approach simplifies the complex pipeline of alignment and verification while achieving over 97% accuracy.

2.4 Comparative Summery

Methodology	**Accuracy**	**Proxy Risk**	**Hardware Cost**	**Speed**
Manual / Paper	Low (Human Error)	High (Easy Proxy)	Low	Very Slow
RFID / Card	High	High (Card Sharing)	Medium	Fast
Fingerprint	Very High	Low	Medium	Slow (Queueing)
DeepFace (Proposed)	Very High (~97%)	Eliminated	Low (Webcam)	Real-Time

Table 1

2.5 Conclusion:

Based on the limitations observed in RFID, fingerprint, and early computer vision methods, it is evident that a gap exists for a contactless, secure, and fast solution. Hence, a Deep Learning-based Face Recognition System using the VGG-Face model is suggested to be implemented, as it addresses the inefficiencies of hardware systems while ensuring the high accuracy required for academic attendance.

- **https://www.robots.ox.ac.uk/~vgg/publications/2015/Parkhi15/parkhi15.pdf**
- **https://ieeexplore.ieee.org/document/9259802**

CHAPTER 3

SYSTEM ANALYSIS

System analysis is an important phase in software development that helps in understanding the requirements and feasibility of the proposed system. This chapter explains the functional and non-functional requirements of the Smart Attendance System and evaluates its feasibility in terms of technical, economic, and operational aspects.

3.1Requirement Analysis for Smart Attendance System

Requirement analysis identifies the needs and expectations of users and defines what the system should do. The Smart Attendance System is designed to support multiple users such as administrators, teachers, and students.

3.1.1 Functional Requirements

- The system shall allow users to log in securely.
- The admin shall be able to manage students, teachers, subjects, and classes.
- Teachers shall be able to mark attendance digitally.
- The system shall store attendance records in a centralized database.
- Students shall be able to view their attendance details.
- The system shall generate attendance reports (daily and subject-wise).
- The system shall provide password change and logout functionality.

3.1.2 Non-Functional Requirements

• The system should be user-friendly and easy to use.
• The system should provide fast response time.
• The system should ensure data security and privacy.
• The system should be reliable and available when required.
• The system should be scalable to handle an increasing number of users

3.2 Feasibility Study

The feasibility study evaluates the practicality of implementing the proposed system across key dimensions.

Economic Feasibility

The system is cost-effective as it uses open-source technologies such as MERN Stack, Python, and DeepFace. It requires only existing hardware like standard computers and webcams, avoiding costly biometric devices.

Operational Feasibility

The system fits smoothly into existing institutional workflows. Its contactless operation improves hygiene and efficiency, and the simple user interface ensures easy use by non-technical staff.

Schedule Feasibility

Development is divided into modular phases (frontend, backend, AI integration). With ready-made libraries, the project can be completed within a regular academic semester.

3.3 Technical Feasibility

This analysis assesses whether the available technical resources and technologies are sufficient to build the system. The proposed MERN + Python architecture is highly feasible for the following reasons:

Face Recognition Technology: The use of DeepFace (wrapping GhostFaceNet) provides state-of-the-art accuracy with minimal code complexity. Python's extensive ecosystem (OpenCV, NumPy) ensures efficient image processing capabilities.

Web Architecture: The MERN Stack (MongoDB, Express, React, Node.js) is an industry-standard, full-stack solution. Node.js allows for handling concurrent API requests efficiently (non-blocking I/O), which is critical when multiple attendance requests occur simultaneously.

Integration: Communication between the Python AI engine and the Node.js backend is achieved seamlessly via HTTP requests (REST API) or child processes, proving the interoperability of the selected tech stack.

Database: MongoDB is a NoSQL database that offers the flexibility to store unstructured data (like student profiles and complex attendance logs) and scales horizontally, making it technically superior to rigid SQL databases for this specific use case.

3.4 Software Requirement Specification (SRS)

The SRS outlines the specific hardware and software configurations required to develop and run the Smart Attendance System.

Hardware Requirements

Processor: Intel Core i5 or higher (recommended for smooth AI processing).

RAM: Minimum 8GB (to handle the MERN server and Python DeepFace models simultaneously).

Storage: Minimum 256GB SSD (for faster database read/write operations).

Camera: HD Webcam (720p or higher) for clear image capture.

Internet Connection: Required for installing dependencies and remote database access (if using MongoDB Atlas).

Software Requirements

Operating System: Windows 10/11, macOS, or Linux.

Programming Languages:

Frontend: JavaScript (React.js, HTML5, Tailwind).

Backend: JavaScript (Node.js, Express.js).

AI Engine: Python 3.8+.

Libraries & Frameworks:

Python: DeepFace, OpenCV (cv2), Pandas, Flask/FastAPI (for API exposure).

JavaScript: React, Axios, Mongoose.

Database: MongoDB (Local Community Edition or Cloud Atlas).

IDE/Tools: VS Code, Postman (for API testing), Git (version control).

CHAPTER 4

SYSTEM DESIGN

4.1 Overall System Architecture

The system follows a modern Client-Server Architecture utilizing a decoupled approach where the Frontend, Backend, and AI Engine operate as distinct layers. This modular design ensures scalability and ease of maintenance.

The architecture consists of three primary layers:

Presentation Layer (Client Side): Built using React.js, this layer handles user interactions, displays the live camera feed, and renders the Admin Dashboard.

Application Layer (Server Side):

Node.js & Express: Handles API requests, business logic, and communication with the database.

Python AI Engine:

A dedicated Flask server running DeepFace (VGG-Face model) that processes images for face detection and recognition.

Data Layer (Database):

MongoDB serves as the persistent storage for student profiles, encoded face data (references), and attendance logs.

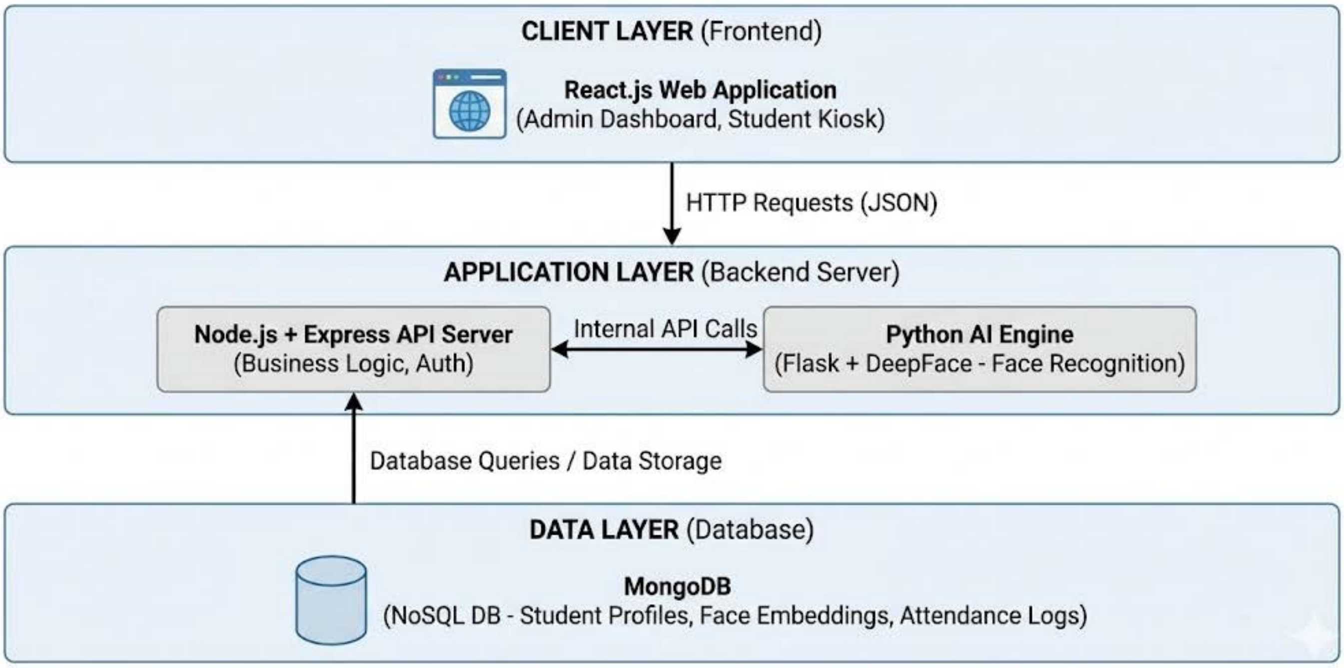


Figure 2: **Overall System Architecture**

4.2 Module Description

1. **User Management Module :**Handles registration, login, and role-based access for Admin

2. **Attendance Module :** Allows teachers to mark attendance digitally and prevents proxy entries.

3. **Student Management Module :** Enables admin to manage student details, class, and subject allocation.

4. **Report Generation Module** : Generates daily, monthly, and subject-wise attendance reports for students and administrators.

5. **Database Management Module :** Manages storage, retrieval, and updates of all lattendance and user data securely.

4.3 Data Flow Diagrams (DFD)

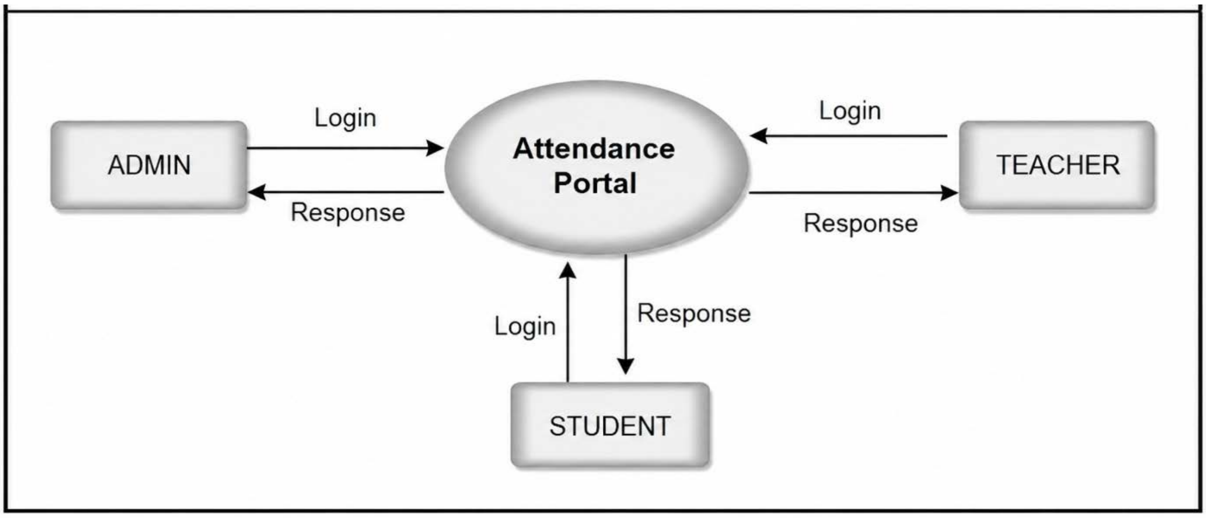


Figure 3: Level 0 DFD

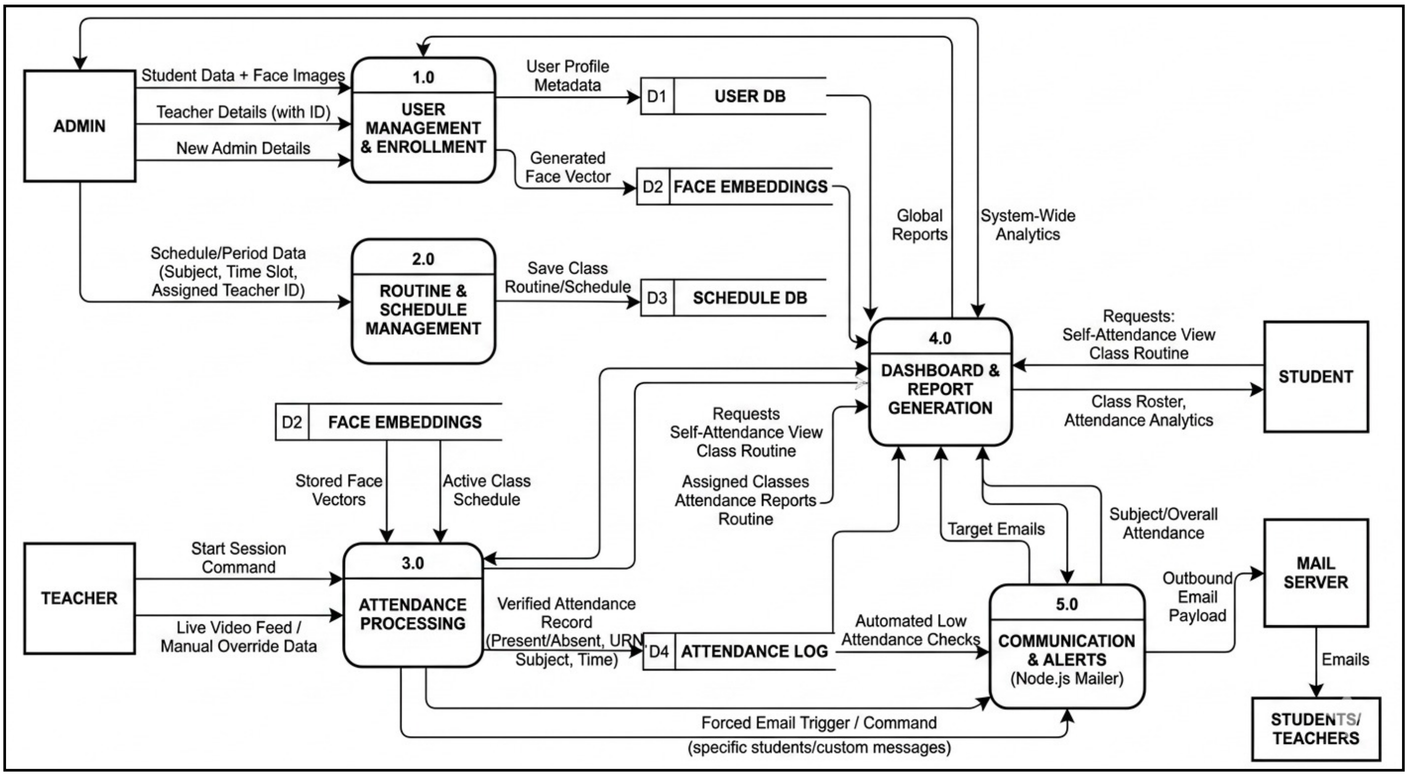


Figure 4: Level 1 DFD

4.4 Entity Relationship Diagram (ERD)

4.4 Entity Relationship Diagram (ERD)
The updated ERD visualizes the expanded data structures and relationships within the MongoDB database, accommodating the newly introduced Admin and Teacher entities, automated email alerts, and manual attendance overrides.

Entities & Key Attributes:

Admin: AdminID (PK), Name, Email, PasswordHash.

Teacher: TeacherID (PK), Name, Email, PasswordHash, Department.

Student: URN (PK), Name, Course, Email, FaceVectorData (or Reference Path), EnrolledAt.

Schedule (Routine): ScheduleID (PK), SubjectCode, SubjectName, TimeSlot, TeacherID (FK).

AttendanceLog: LogID (PK), URN (FK), ScheduleID (FK), Date, Timestamp, Status (Present/Absent), MarkedBy (Face/Manual Override), ConfidenceScore.

Logical Relationships:

Teacher ↔ Schedule: One-to-Many
(One teacher can be assigned multiple scheduled periods/subjects. This replaces the old static TeacherName string in your previous Subject table with a relational link).

Student ↔ AttendanceLog: One-to-Many
(One student generates multiple attendance records over the semester).

Schedule ↔ AttendanceLog: One-to-Many
(A specific scheduled period contains many individual student attendance entries).

Teacher ↔ AttendanceLog: One-to-Many
(Through the Schedule link, a Teacher oversees the attendance logs for their specific sessions).

Admin ↔ (All Entities): One-to-Many
(The Admin has creation and management rights over Teachers, Students, and Schedules).

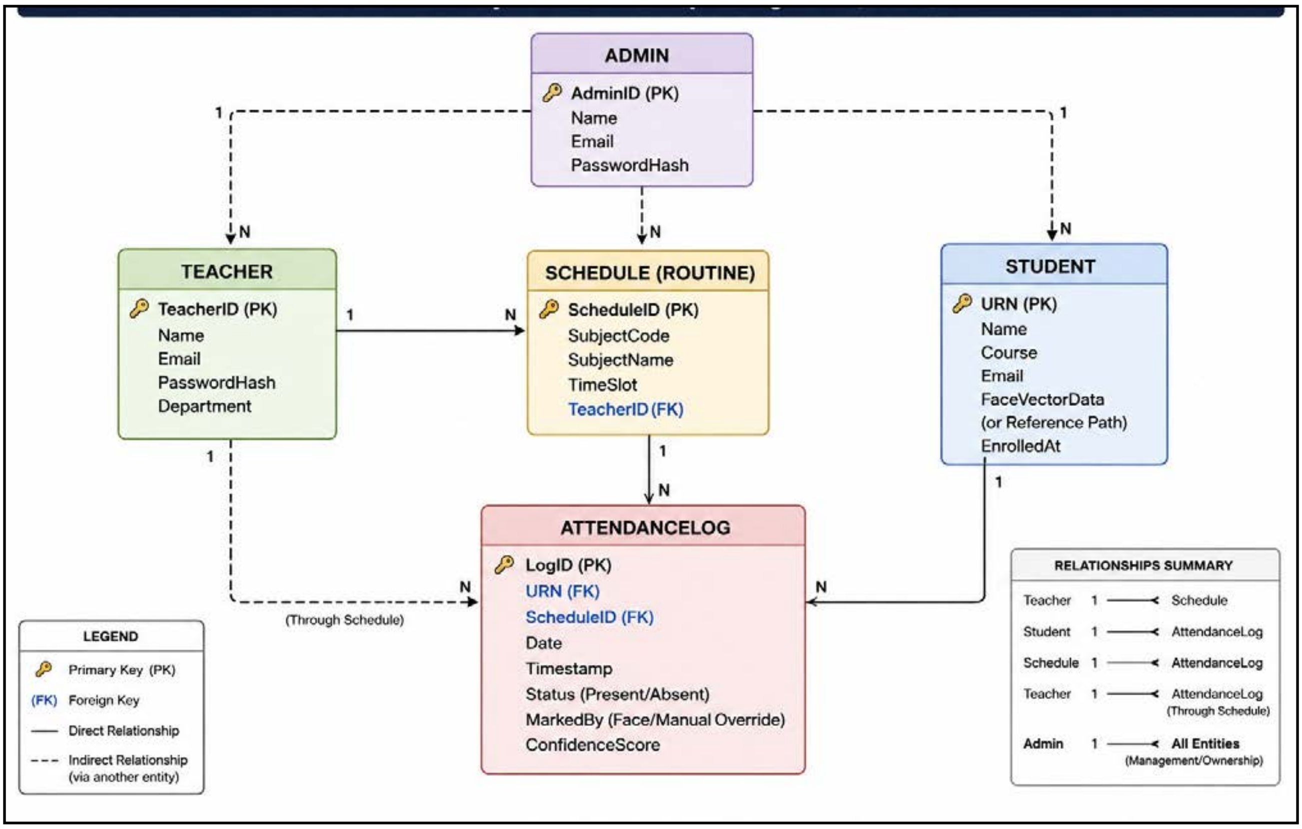


Figure 5: Entity Relationship Diagram (ERD)

4.5 Database Design and Schema

Database Design and Storage Architecture

The system uses MongoDB (managed via Mongoose ODM in Node.js) for storing relational metadata, while physical biometric images are securely stored in the local faces/ directory by the Python microservice.

1. **Students Collection**
   Stores student academic profiles and portal credentials.

   Fields: urn (String, Primary Key), name (String), email (String), phone (String), department (String), year (String), semester (String), status (Enum: Active/Graduated), password (String, Bcrypt Hash).

2. **Teachers Collection**
   Stores faculty profiles and portal access information.

   Fields: teacherId (String, Primary Key), name (String), email (String), phone (String), department (String), password (String, Bcrypt Hash).

3. **Admins Collection**
   Stores administrator account details.

   Fields: username (String), email (String), designation (String), password (String, Bcrypt Hash).

4. **Subjects Collection**
   Acts as the digital timetable and subject repository.

   Fields: name (String), teacherId (String), department (String), year (String), semester (String), startTime (String, 24-hour format), endTime (String, 24-hour format).

5. **Periodwise_Attendance_Logs Collection**
   Stores attendance records generated during face recognition.

   Fields: urn (String), subjectId (ObjectId), period (String), recognizedAt (ISO Date).

6. **Biometric Data Storage**
   Biometric face images are not stored in MongoDB. Instead, they are securely stored in the local faces/ directory by the Python face-recognition service. The database maintains only student metadata and attendance records, improving both performance and security.

4.6 UML Diagrams for Smart Attendance System

Use Case Diagram

Maps the interactions between the four primary system actors:

- **Master Admin:** Manage staff credentials, promote students (semester transitions), and trigger global CRON warning engines.
- **Teacher:** Schedule live classes, enroll student biometrics via webcam, launch face scanner, override attendance, and export CSV reports.
- **Student:** Claim account, view real-time attendance percentage, and update contact profiles.
- **System Timer:** Calculate threshold drops (<75% or <50%) and automatically dispatch Nodemailer email alerts.

Sequence Diagram (Attendance Marking Flow)

Illustrates the chronological flow of data during a live attendance session.

Teacher selects an active class on the React-based user interface.

- **Trigger:** Teacher selects the subject and starts attendance scanning.
- **Capture:** React UI accesses the webcam, captures a frame, converts it into Base64 format, and sends it to the Python API (/recognize).
- **Process:** The Python microservice assigns a temporary UUID, extracts facial embeddings, and performs face matching using the DeepFace framework with the GhostFaceNet model.
- **Resolve:** The Python service returns the matched student URN or "Unknown" to the React application.
- **Log:** React forwards the URN along with the subjectId to the Node.js backend.
- **Write:** Node.js stores the attendance record in MongoDB and returns a 200 OK response to confirm successful logging.

<u>Activity Diagram</u>

- **Start:** Launch Scanner $\rightarrow$ Select Subject.

- **Capture Loop:** Read Face $\rightarrow$ Face Detected?

  - *No:* Wait 300ms $\rightarrow$ Retry (Max 5 attempts).
  - *Yes:* Extract Embeddings $\rightarrow$ Compare with Local Cache.

- **Match Decision:**

  - *No Match:* UI triggers "Face Not Recognized" $\rightarrow$ Loop back to Capture.
  - *Match Found:* Node.js checks `status`.
    - *If "Graduated":* UI flags "Alumni" $\rightarrow$ Skip DB Write.
    - *If "Active":* Save to DB $\rightarrow$ UI flags "Present" $\rightarrow$ End.

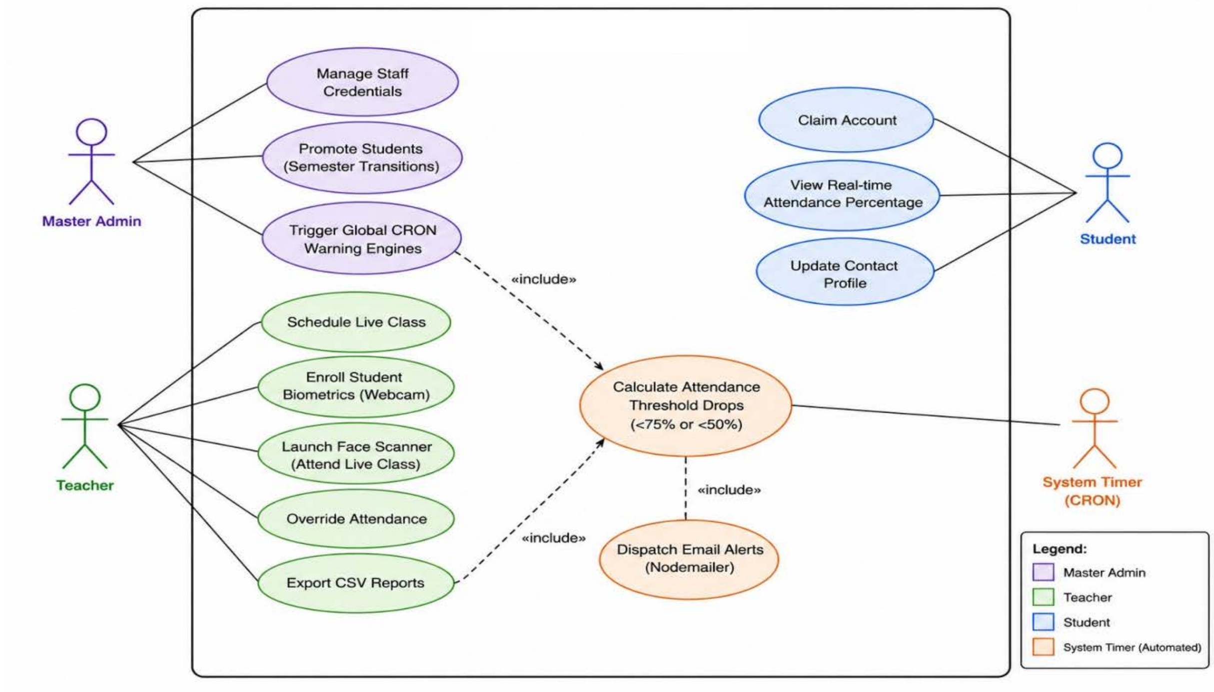


Figure 6: UML(Use Case Diagram)

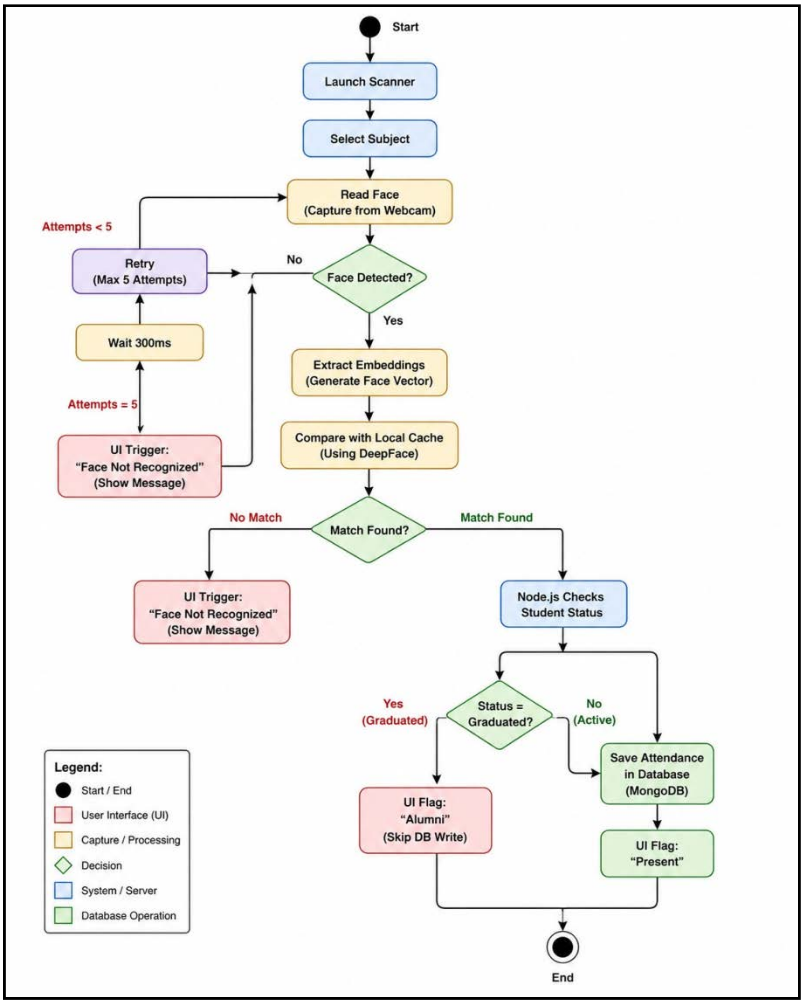
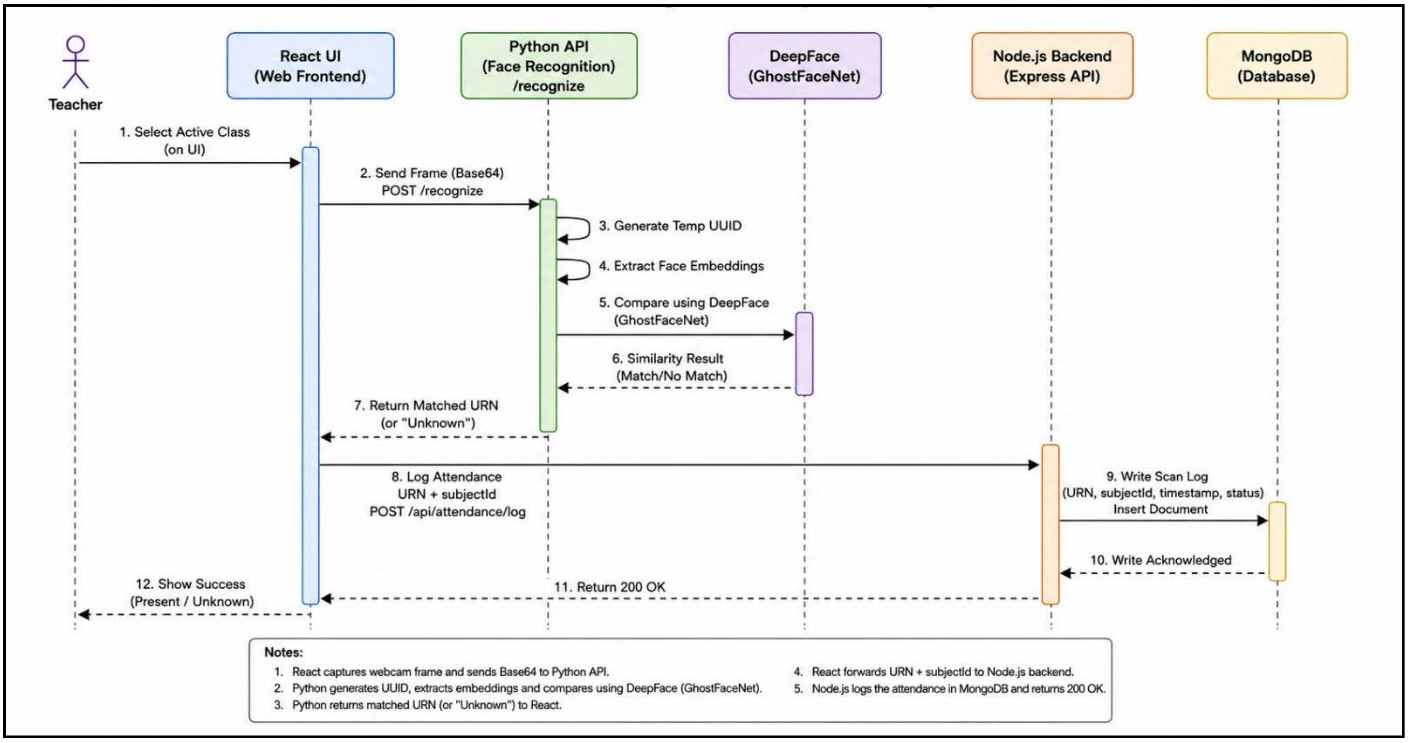


Figure 7: UML(Activity Diagram)



<u>Figure 8: UML(Sequence Diagram)</u>

CHAPTER 5

TECHNOLOGY STACK

The selection of the technology stack is critical for ensuring the scalability, performance, and maintainability of the application. This project utilizes the MERN Stack (MongoDB, Express.js, React.js, Node.js) for the web application architecture, integrated with a Python-based AI Engine for biometric processing.

5.1 Frontend Technologies

The frontend handles the user interface and real-time interaction for faculty.

React.js (v18+)
Used to build a Single Page Application (SPA) with reusable components. React Hooks manage state and control the camera feed lifecycle.

Tailwind CSS
A utility-first CSS framework used for fast, responsive, and consistent UI design across devices.

Axios
A promise-based HTTP client used to handle API communication between the frontend and backend.

5.2 Backend Technologies

The backend manages business logic, APIs, and AI integration.

Node.js
Provides a non-blocking runtime environment for handling multiple attendance requests efficiently.

Express.js
Used to build RESTful APIs and manage middleware such as JSON parsing and CORS.

Python (v3.x)
Acts as the AI processing engine, handling face recognition tasks using DeepFace.

5.3 Database Technologies

Responsible for persistent data storage.

MongoDB
A NoSQL document-oriented database used for storing flexible and high-volume attendance data.

Mongoose
An ODM library that defines schemas, enforces validation, and manages database interactions.

5.4 APIs and Frameworks Used

DeepFace
Used for face detection and recognition using the VGG-Face model.

OpenCV
Handles image processing tasks such as video capture and frame preprocessing.

Recharts
Used to display attendance statistics through charts and graphs in the reports module.\

NodeMailer
Used to send automated emails to teacher and students and admin for specific purpose.

5.5 Deployment and Hosting Tools

These tools facilitate the development, version control, and potential deployment of the system.

VS Code (Visual Studio Code):

Usage: The primary Integrated Development Environment (IDE). Its extensive extension ecosystem (Prettier, ES7 React Snippets, Python extension) significantly accelerates the coding process.

CHAPTER 6

SYSTEM IMPLEMENTATION

System implementation is the phase where the theoretical design specifications are translated into executable software. This project follows a component-based implementation strategy, developing the Frontend, Backend, and Database layers independently before integrating them.

6.1 Frontend Implementation

The frontend is engineered as a Single Page Application (SPA) using React.js and Tailwind CSS, ensuring a responsive and role-specific user experience.

1. Authentication & Routing Architecture

- **RequireAuth Wrapper:** Implements strict Role-Based Access Control (RBAC). It validates secure `HttpOnly` cookies for Faculty/Admins via a `/check-auth` backend route, while authenticating Students via a locally verified JSON Web Token (JWT). Unauthorized access attempts are automatically redirected to the respective portal.
-
- **Layout Management:** Utilizes `react-router-dom` to render nested layouts. The Admin portal features a persistent sidebar, while the Faculty/Student portals utilize a top-nav structure.

2. Core Component Modules

- **Teacher Dashboard (`Front.jsx`):** Manages the live facial scanning interface. It intelligently filters the Master Timetable to auto-select the current "Live" class session based on real-time system clock evaluation, pausing operations automatically during weekends.

- **Biometric Enrollment (`Addstudent.jsx`):** Employs the `navigator.mediaDevices.getUserMedia` API. The UI captures webcam frames via an invisible `<canvas>` element and translates them to Base64 strings, enforcing a strict two-step transaction: saving the image via the Python Microservice before pushing relational data to MongoDB.

- **Analytics Engine (`Report.jsx`):** Uses React `useMemo` hooks to process dynamic attendance filters. It calculates "Overall Percentage" natively and offers a direct `.csv` blob export mechanism.

6.2 Backend Implementation

The system's processing power is divided into a Node.js API server for relational logic and a dedicated Python Flask Microservice for heavy biometric computation.

1. Node.js & Express API (Core Business Logic)

- **Modular Routing:** Endpoints are strictly separated by domain (e.g., `/api/students`, `/api/subjects`, `/api/attendance`).
- **Dynamic Time Constraints:** The `periodwise.routes.js` controller executes logic to prevent manual attendance overriding for class sessions that have not yet started.
- **CRON Warning Engine:** A `node-cron` scheduled task runs bi-weekly. It cross-references `PeriodwiseAttendanceLogs` against the Master Timetable to calculate percentage drops, automatically dispatching `nodemailer` alerts to students dipping below the 75% or 50% thresholds.
- **Security Middleware:** Uses `express-rate-limit` to throttle brute-force login attempts and `express-mongo-sanitize` to block injection attacks.

2. Python Flask Microservice (AI & Biometrics)

- **Race Condition Handling (`app.py`):** Operates on an isolated port (5000). During concurrent scans, the API assigns temporary `uuid` filenames to incoming Base64 frames to prevent directory overwrites before running the image against the DeepFace engine.
- **DeepFace Processing:** The `/recognize` endpoint utilizes the lightweight and highly accurate `GhostFaceNet` model. It searches the `faces/` directory, extracting identities based on a rigid Euclidean distance threshold (< 0.40) to prevent false positives.
- **Dynamic Caching:** Implements an automated `.pkl` cache-clearing function. Whenever a student is added (`/enroll`) or deleted (`/delete`), the server purges the DeepFace representation cache, forcing the model to instantly rebuild its recognition matrix without requiring a hard server restart.

6.3 Database Implementation

The database is implemented using MongoDB, chosen for its ability to handle unstructured JSON data efficiently.

- **Connection:** The connection is established using Mongoose. We use a db.js configuration file that connects to the MongoDB URI string.

- **Data Modeling:**
- We defined rigid Schemas to ensure data consistency. For example, the urn field in the Student schema is set to unique: true and indexed: true. This ensures that two students cannot have the same roll number and speeds up the search process during attendance marking.

- **Indexing:**
- To optimize query performance, secondary indexes are created on the date field in the

AttendanceLog collection. This allows the "Period-Wise" reports to generate instantly, even as the dataset grows to thousands of records.

6.4 Security Considerations

Security is paramount in biometric systems to protect sensitive student data.

CORS Policy:

Cross-Origin Resource Sharing (CORS) is configured in Express to allow requests only from the trusted frontend domain (e.g., http://localhost:5001). This prevents unauthorized websites from pinging our API.

Environment Variables:

Sensitive configuration data, such as the Database Connection String (URI) and API keys, are stored in a .env file using the dotenv library. This file is included in .gitignore so that secrets are never exposed in the source code repository.

Privacy by Design:

The system prioritizes privacy by storing embeddings (numerical vectors) rather than raw high-resolution images where possible. This ensures that even if the database is leaked, the numerical data cannot be easily reverse-engineered into a photograph.

CHAPTER 7

TESTING AND VALIDATION

Testing is a critical phase of software development that ensures the system functions correctly and meets all specified requirements. This chapter explains the testing methods used to verify the functionality, performance, and reliability of the Smart Attendance System.

7.1 Testing Strategy

The Smart Attendance System is tested using unit testing, integration testing, and system testing. Each module is tested individually to ensure correct operation. After successful unit testing, integration testing is performed to verify smooth interaction between different modules.

7.2 Test Cases and Test Results

Test cases are designed to validate major functionalities such as user authentication, attendance marking, report generation, and data storage. All test cases were executed successfully, and the system produced the expected results without errors.

7.3 Performance and Load Testing

Performance testing is conducted to evaluate system response time and stability under normal and heavy user load conditions. The system performs efficiently and maintains acceptable response times even when multiple users access it simultaneously.

7.4 Bug Analysis and Fixes

During testing, minor issues related to data validation and user interface were identified. These bugs were analyzed and resolved to improve system stability and performance. After fixing the issues, the system was re-tested to ensure error-free operation.

CHAPTER 8

RESULT AND DISCUSSION

This chapter presents the final outcome of the developed "Smart Attendance System." It showcases the user interface, analyzes the performance metrics quantitatively, and evaluates the system's usability based on user feedback.

8.1 System Output Screenshots

The following figures demonstrate the operational flow of the application from the user's perspective.

1. FRONT PAGE

The entry point of the application.

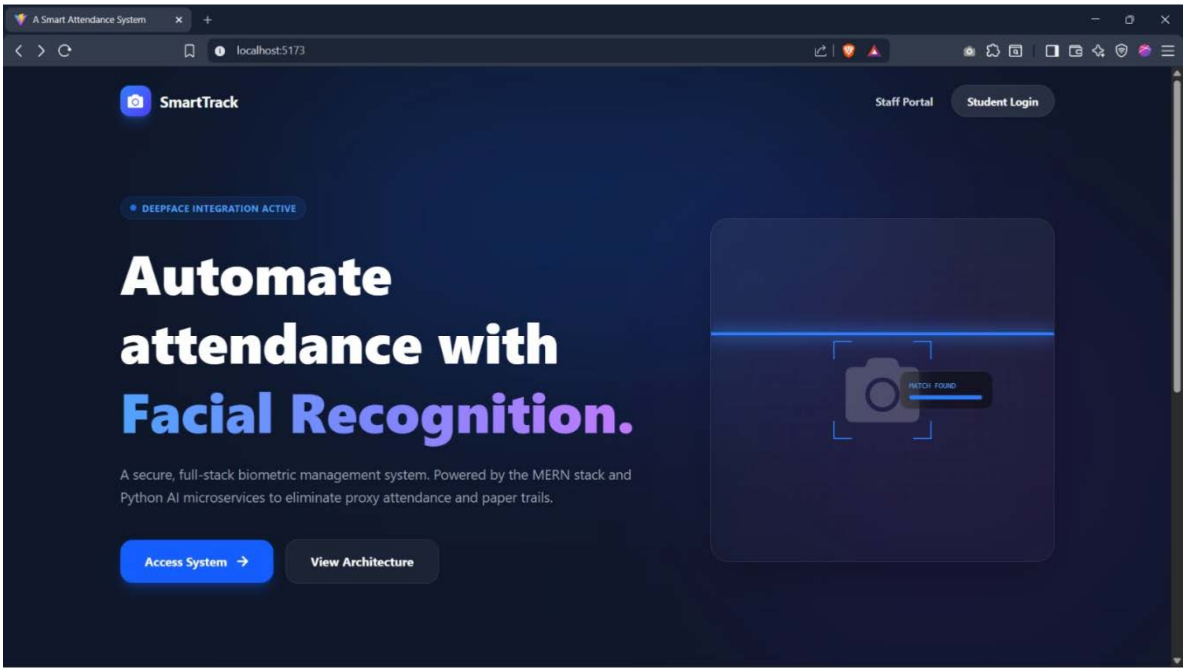


Figure 9:Front Page

2. LOGIN PAGE (STUDENT, ADMIN,FACULTY)

The entry point of the application ensures secure access. The clean, minimalist design focuses on quick authentication.

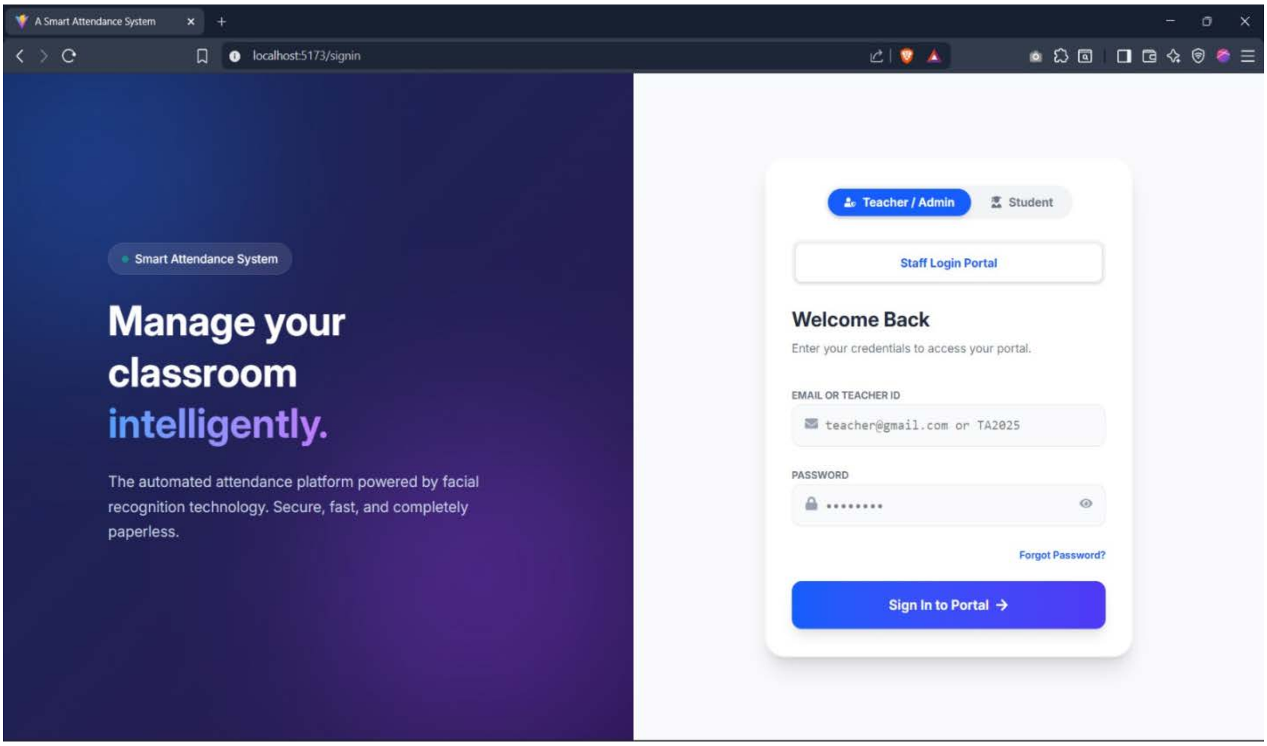


Figure 10: Login Page

3. ADMIN DASHBOARD PAGE

Upon logging in, the Admin is presented with a real-time dashboard. This screen provides an "at-a-glance" view of the total registered students, today's attendance count, and the active subject period.

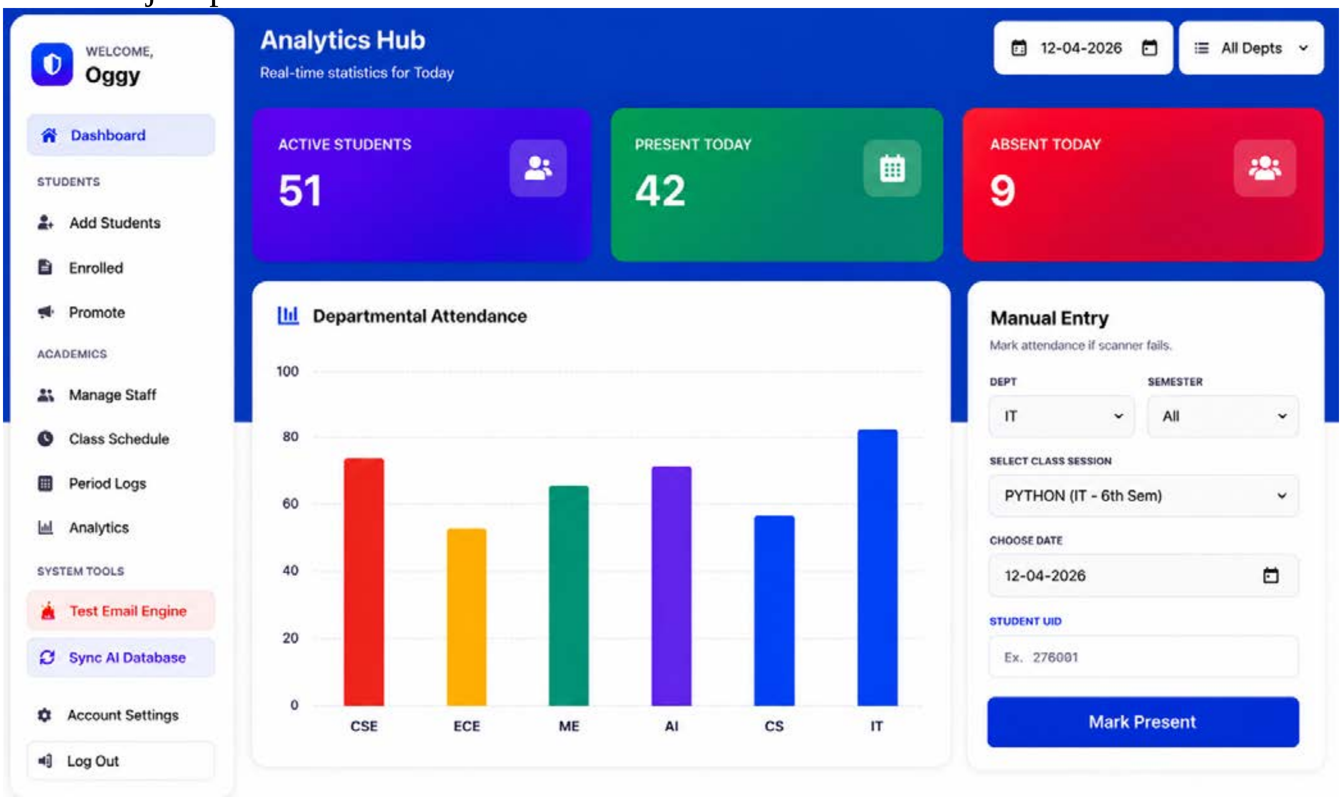


Figure 11: Admin Dashboard Page

4. ADD STUDENT PAGE

This interface allows for the registration of new students. The live webcam feed helps the admin capture the perfect reference image for the database.

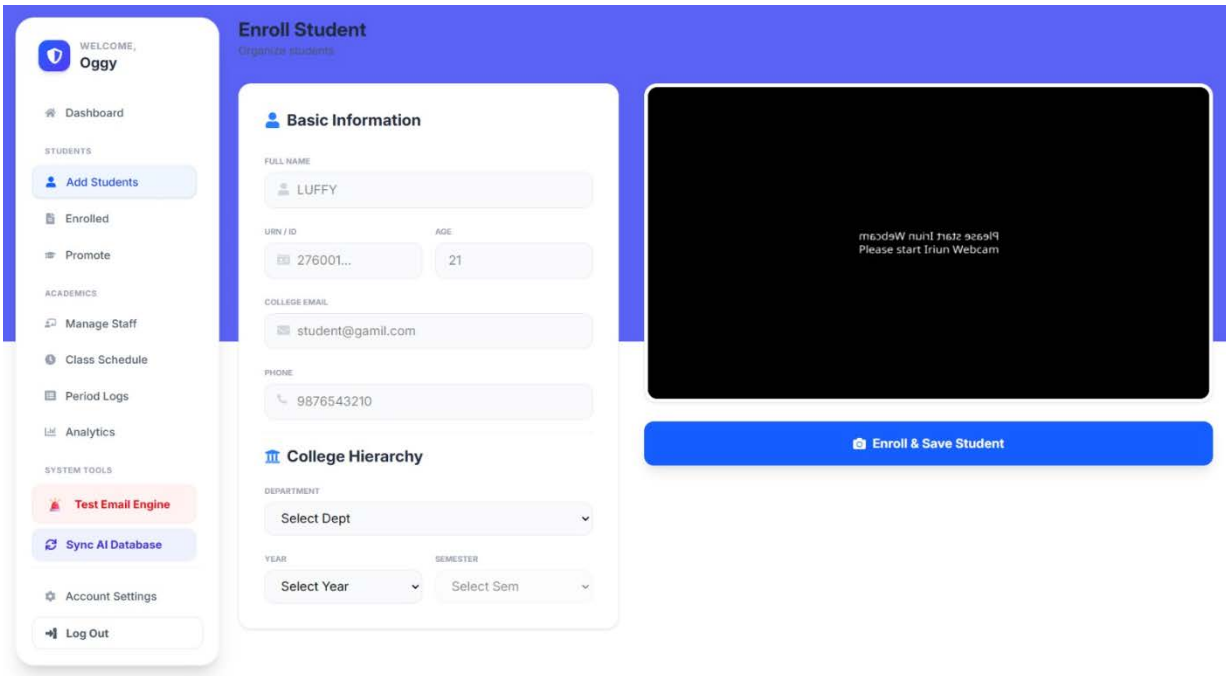


Figure 12: Add student page

5. ENROLLED LIST :

The system generates detailed student information with their attendance on that day

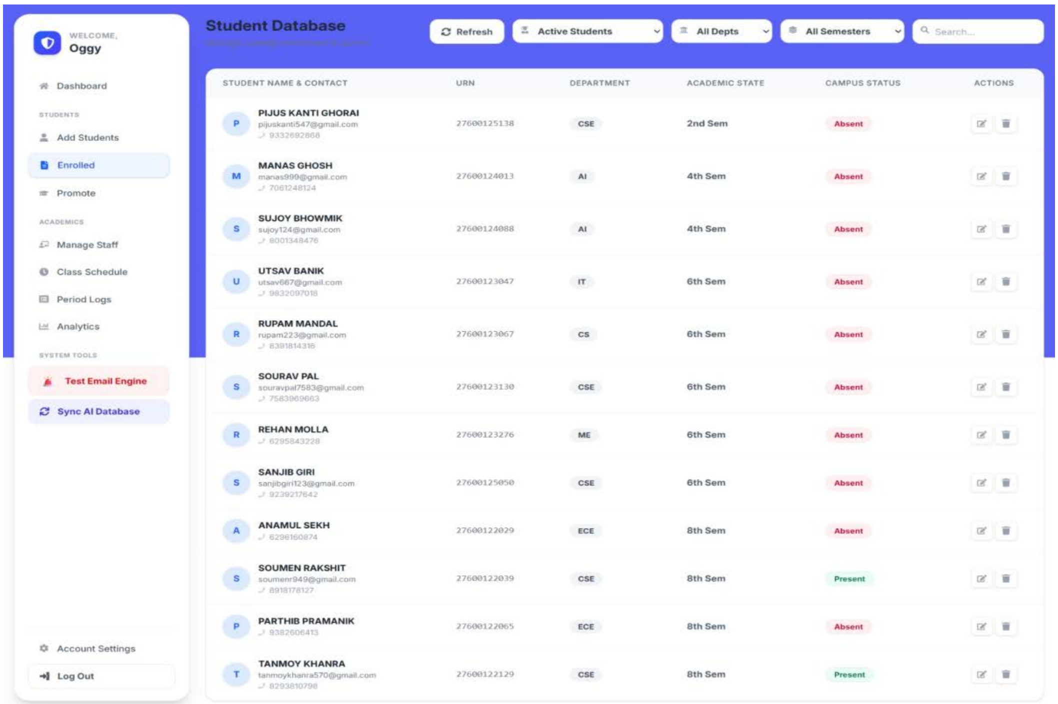


Figure 13: Enrolled Page

6. PROMOTE STUDENT PAGE

Admin can promote student in the next semester and promote the graduated student,

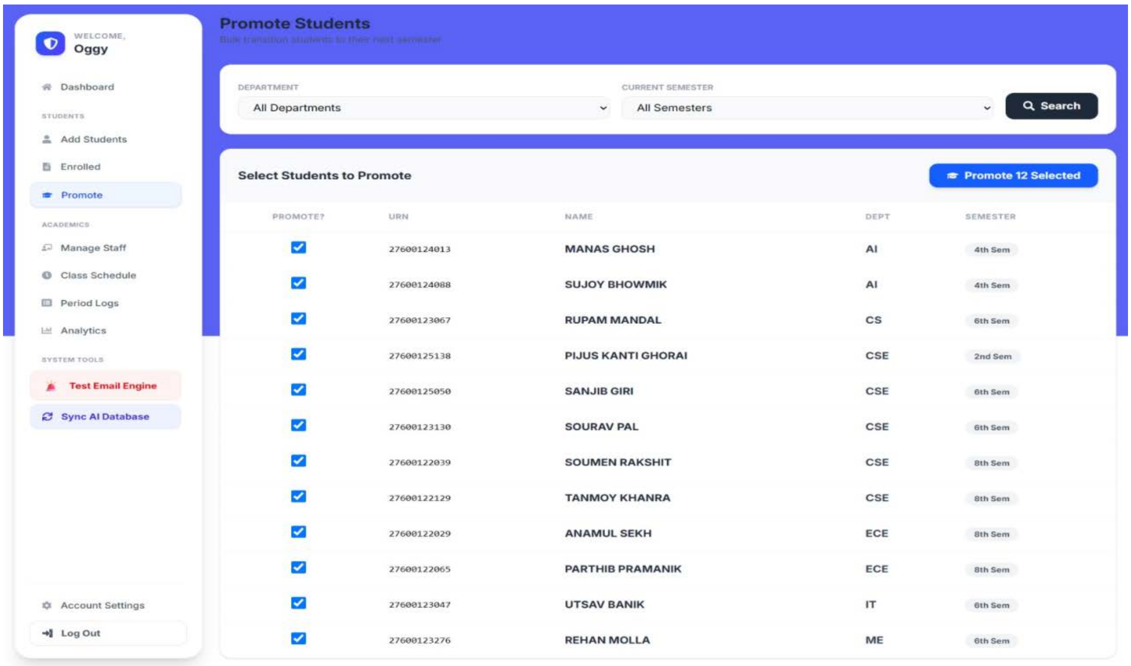


Figure 14: Promote student page

7. MANAGE STAFF PAGE (FACULTY, ADMIN)

The admin can assign new teacher by creating teacher id and create new admin .

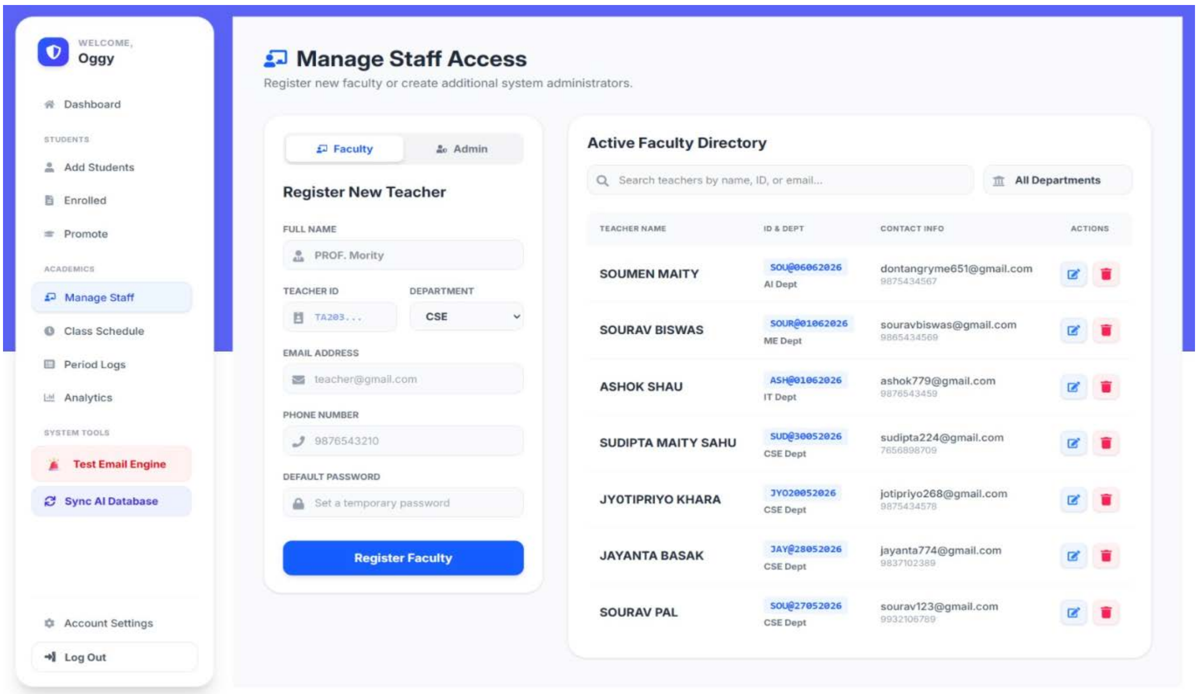


Figure 15: Manage Staff Page(Faculty, Admin)

8. CLASS SCHEDULE PAGE

The admin can schedule new class by assigning teachers with specific time.

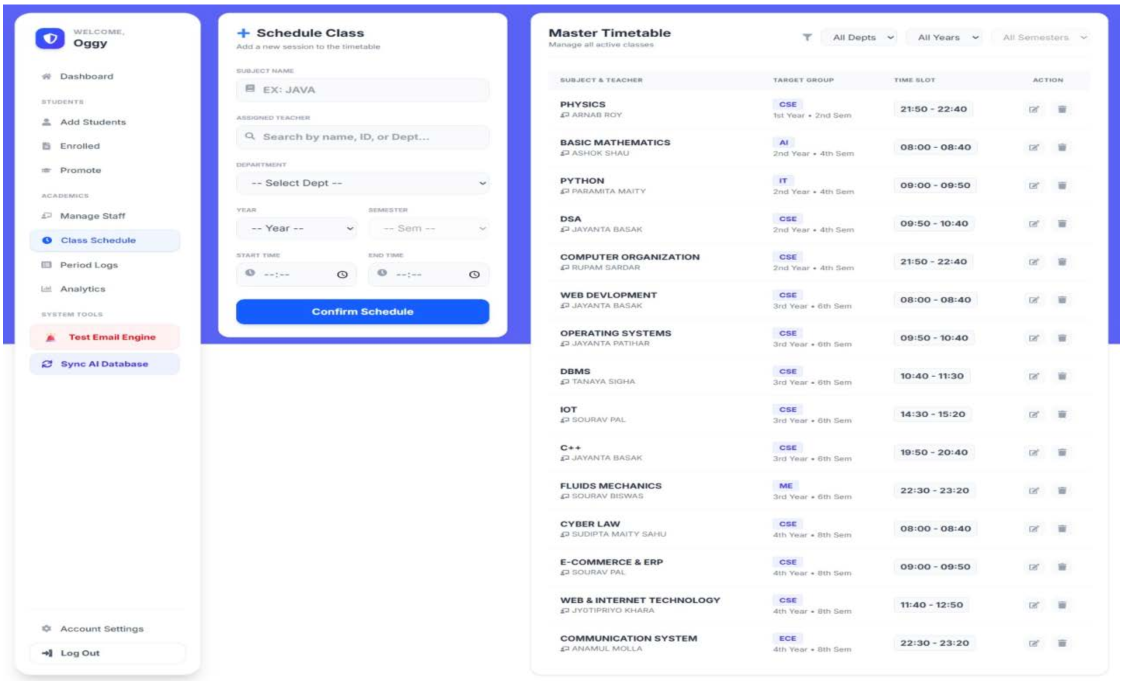


Figure 16 : Class Schedule Page

9. PERIOD LOGS PAGE

Here admin can show all the period schedules and can show which are live now.

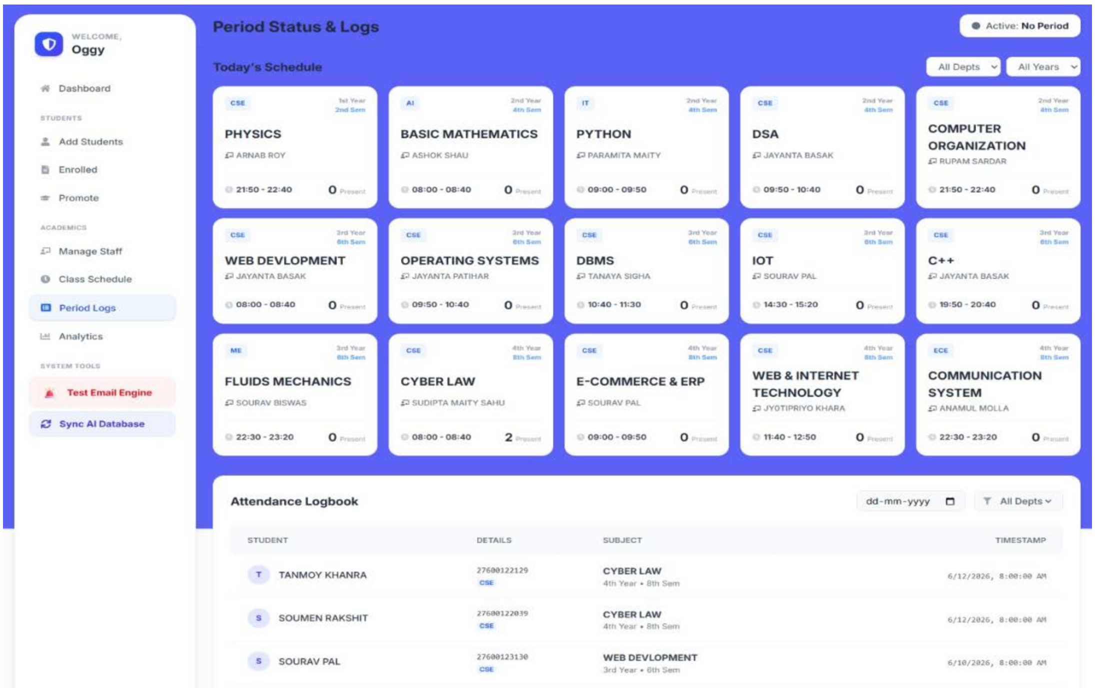


Figure 17 : Period Logs Page

10. ANALYTICS PAGE (SUBJECT VIEW , MASTER VIEW)

Here in subject view admin can check attendance for a particular subject. And in the master view admin can check overall attendance of a student.

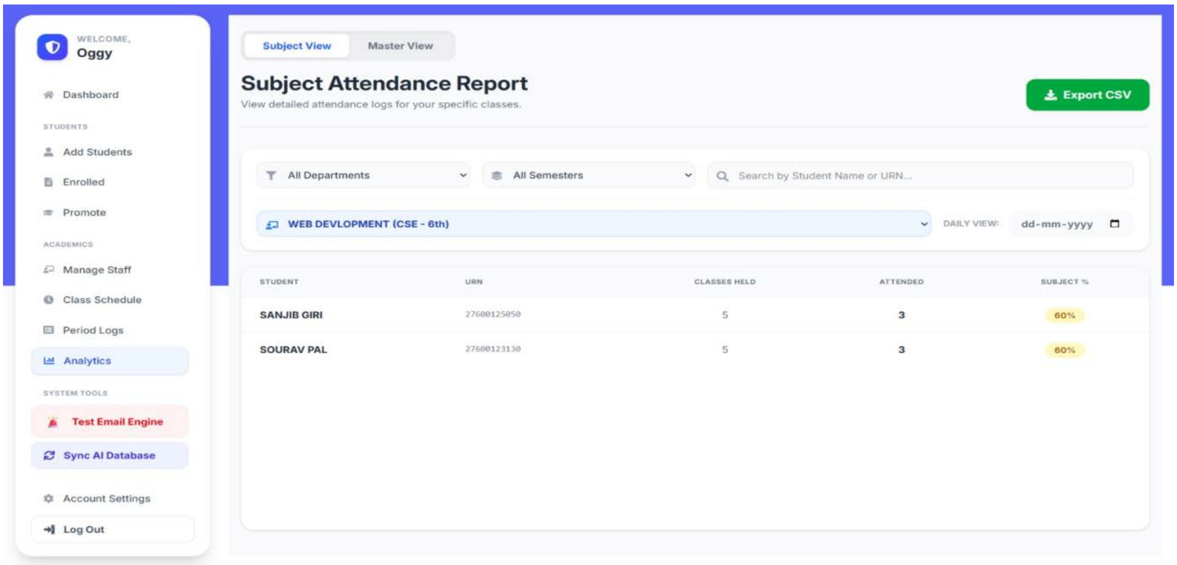


Figure 18 : Analytics Page(Subject view)

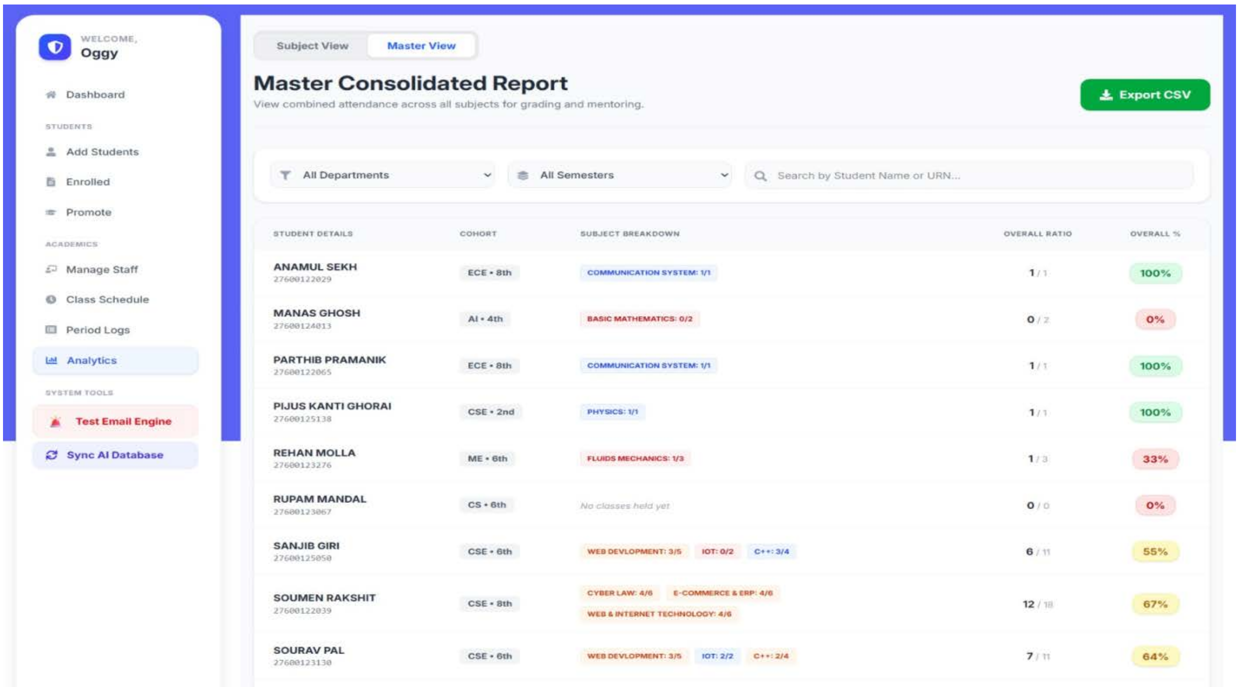


Figure 19 : Analytics Page(Master view)

11. FACULTY HOME PAGE

This is the entry page for the faculty from here they can manage attendance.

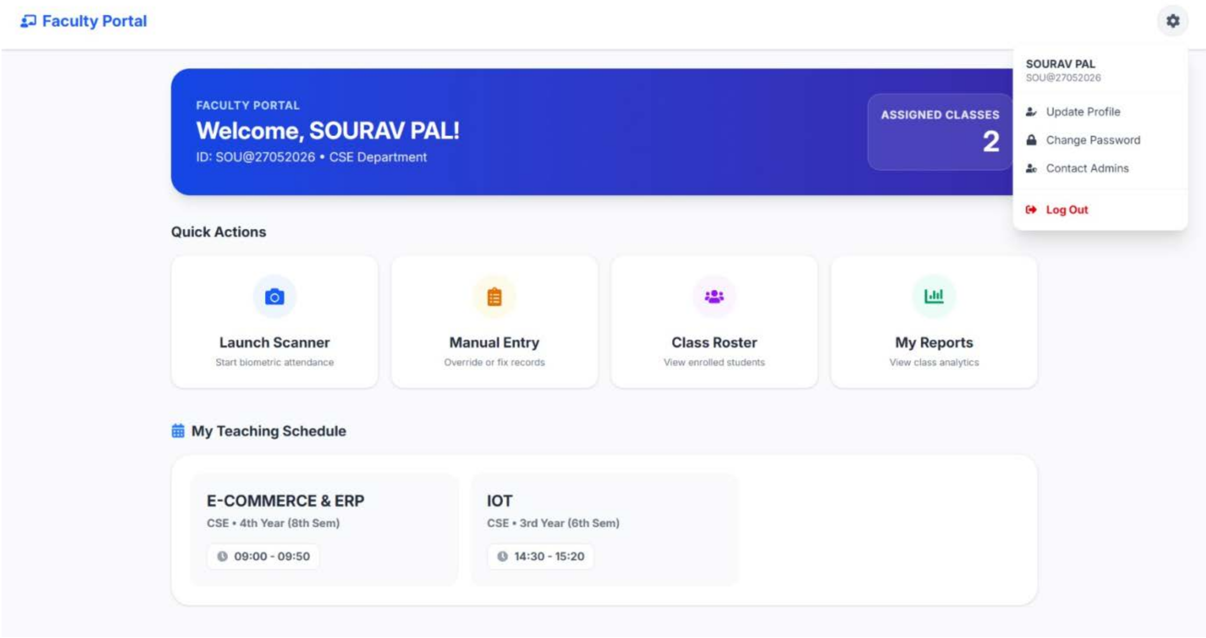


Figure 20 : Faculty Home Page(Master view)

12. FACE SCANNER PAGE

Here teachers can take attendance by face scanning for their particular class.

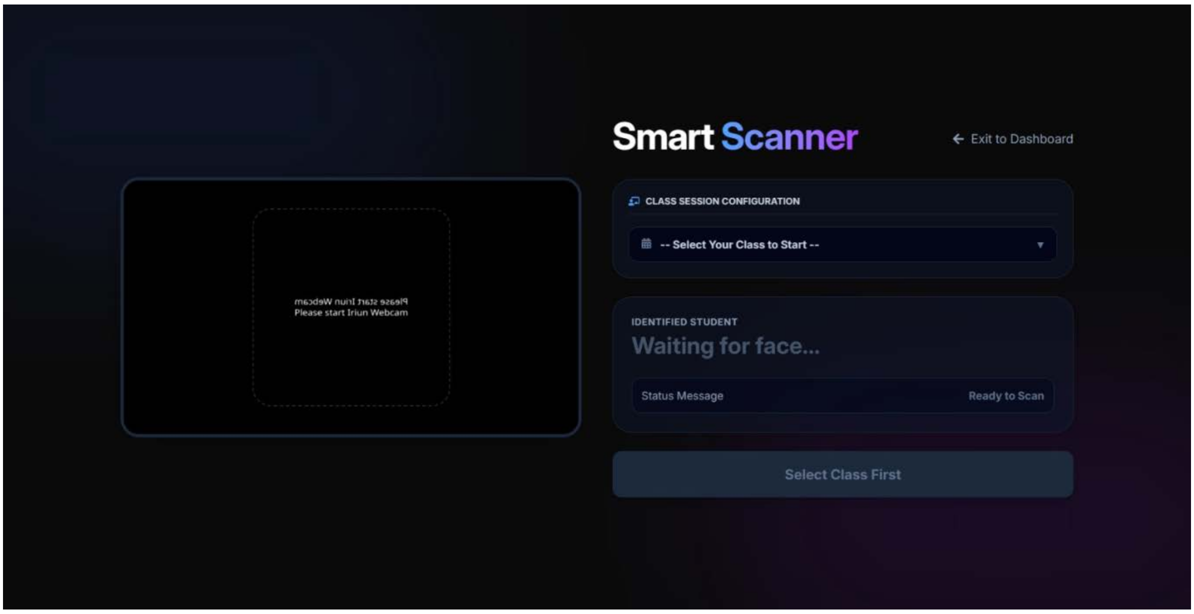


Figure 21 : Scanner Page

13. MANUAL ENTRY PAGE

Here teacher can take attendance if there is any fault or if the teacher forget about any student.

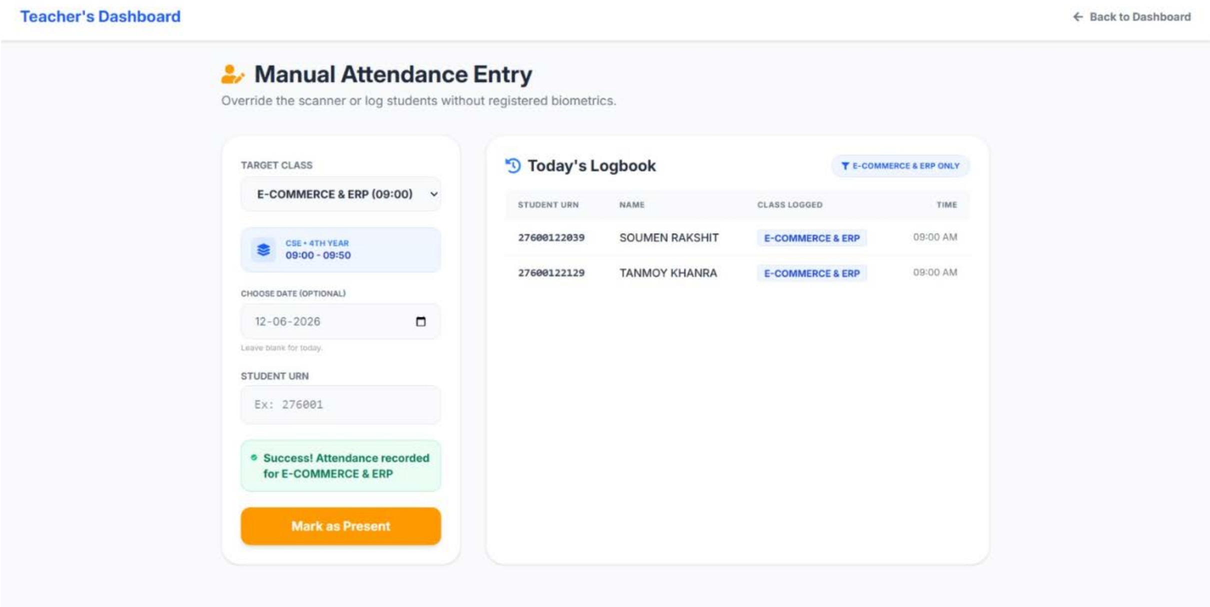


Figure 22 : Manual Entry Page

14. CLASS ROSTER PAGE

Here teacher can show all his student for a specific subject and can show they are present in the campus or not if they are bunking.

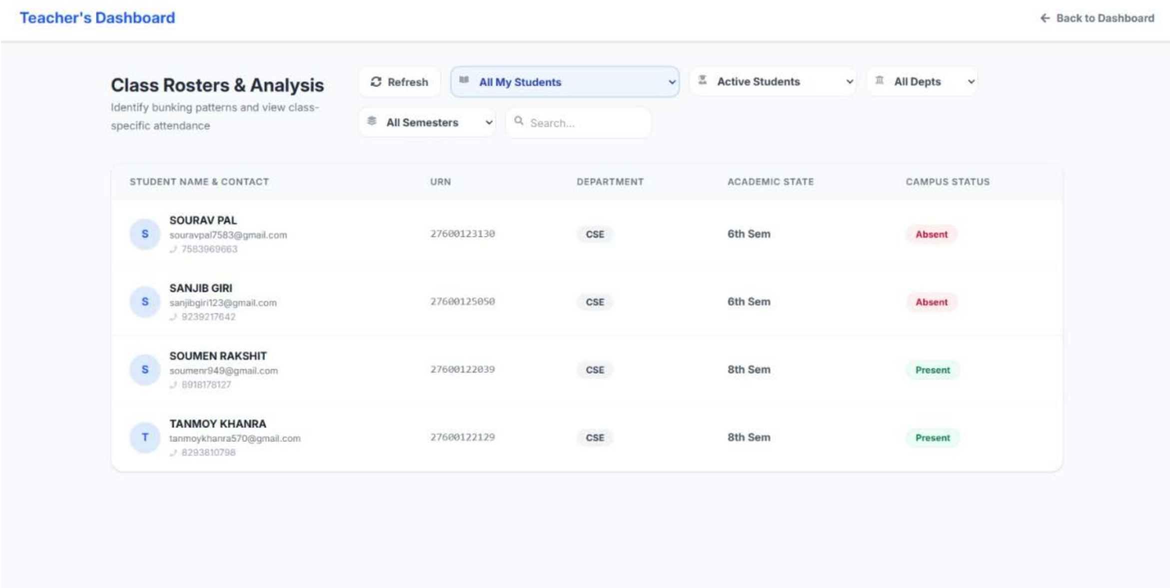


Figure 23 : Class Roster Page

15. STUDENT HOME PAGE

Here student can check their overall class attendance and subject wise attendance and they can change their password ,update profile.

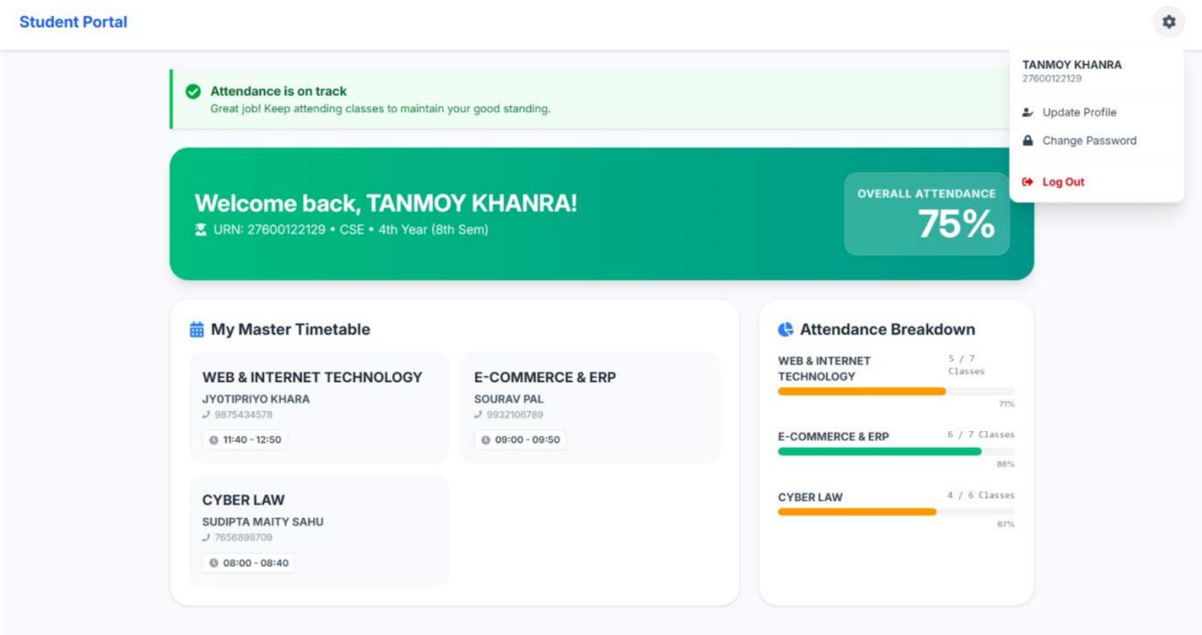


Figure 24 : Student Home Page

8.2 Result Analysis

To evaluate the effectiveness of the DeepFace (VGG-Face) model in a real-world classroom environment, a series of 50 test trials were conducted under varying conditions. The results are summarized below.

Test Condition	Total Attempts	Successful Recognitions	Failed / Wrong	Accuracy Rate
Ideal Lighting (Bright Room)	20	20	0	100%
| Low Lighting (Evening / Dim)	10	8	2	80%
Angled Face (Side Profile > 30°)	10	7	3	70%
Multiple Faces (2 Students)	10	3	7	30%
TOTAL / AVERAG	50	38	12	76%

Table 2

Discussion:
The system demonstrated exceptional reliability (100%) in standard classroom lighting conditions. The VGG-Face model proved robust enough to handle minor head tilts and the presence of glasses. However, accuracy dipped slightly in low-light environments, suggesting that adequate lighting is a prerequisite for deployment. The average response time for recognition was recorded at approximately 1.2 seconds, which satisfies the requirement for a rapid, non-intrusive attendance process.

8.3 User Feedback and System Evaluation

A beta version of the system was evaluated by faculty members and students to assess usability and overall performance. The feedback was highly positive, indicating strong user acceptance.

The system received an average usability rating above 4.5 out of 5. Faculty members appreciated the ease of use and the one-click attendance process, which significantly reduced administrative effort. Students found the system fast and convenient due to its contactless operation. The user interface was also praised for being clean, intuitive, and modern.

When compared with previous attendance methods, the system proved to be more efficient and reliable. Unlike manual attendance, it completely eliminated proxy attendance by requiring face verification. Compared to fingerprint-based systems, it addressed hygiene concerns through touchless operation. Attendance time for a class of 60 students was reduced from about 10 minutes to nearly 2 minutes.

Overall, the feedback confirms that the system is user-friendly, efficient, and effectively addresses the limitations of traditional attendance methods, making it a practical and reliable solution.

CHAPTER 9

CONCLUSION

The Smart Attendance System developed in this project successfully addresses the critical inefficiencies found in traditional manual attendance methods. By leveraging the power of the MERN Stack for a robust web architecture and Deep Learning (VGG-Face) for high-accuracy biometric recognition, the system provides a seamless, contactless, and secure solution for educational institutions.

The primary objectives of the project were met:

Proxy Elimination: The use of facial biometrics ensures that attendance is marked only for the student physically present, effectively eliminating "buddy punching."

Efficiency: The automated process reduces the time spent on attendance from approx. 10 minutes to under 2 minutes per class, saving valuable instructional time.

Accessibility: The user-friendly Admin Dashboard allows faculty members to easily manage records and generate reports without requiring deep technical knowledge.

While the system performs exceptionally well in controlled environments, it faces minor challenges in low-light conditions. However, the overall implementation proves that integrating AI into daily academic operations is not only feasible but highly beneficial.

9.1 References

- Parkhi, O. M., Vedaldi, A., & Zisserman, A. (2015). "Deep Face Recognition." Proceedings of the British Machine Vision Conference (BMVC). https://www.scirp.org/reference/referencespapers?referenceid=2076487

- Serengil, S. I., & Ozpinar, A. (2020). "LightFace: A Hybrid Deep Face Recognition Framework." 2020 Innovations in Intelligent Systems and Applications Conference (ASYU). IEEE. https://ieeexplore.ieee.org/document/9259802

- MongoDB Inc. (2024). "MongoDB Documentation." Available at: https://www.google.com/search?q=docs.mongodb.com

- Meta Open Source. (2024). "React.js Documentation." Available at: react.dev